

THE BLACK CODE JSB

CommerCity Mobile

Especificación de Requerimientos de Software
Sistema de Marketplace Local — Versión 2.0

CÓDIGO DOCUMENTO

SRS-CCM-2026-V3

ESTÁNDAR

IEEE Std 830-1998 / ISO/IEC/IEEE 29148

VERSIÓN

3.0 — Edición Completa

FECHA DE EMISIÓN

Septiembre 2026

ORGANIZACIÓN

THE BLACK CODE JSB

CLASIFICACIÓN

Uso Interno — Sprint 2 / Cierre

PLATAFORMA

Android & iOS (Ionic Capacitor)

BACKEND

Node.js / Express — MySQL 8.0

✦ AUTOR Y RESPONSABLE DEL DOCUMENTO

Jhon Jairo Parra Obando

Tabla de Contenidos

Documento SRS-CCM-2026-V3 — CommerCity Mobile v2.0 — Especificación Completa

1.	Introducción	3
1.1	Propósito del Documento	3
1.2	Alcance del Sistema	3
1.3	Definiciones, Acrónimos y Abreviaturas	4
1.4	Referencias y Estándares Aplicables	4
1.5	Visión General del Documento	5
2.	Descripción General del Sistema	5
2.1	Perspectiva del Producto	5
2.2	Funciones Principales del Producto	6
2.3	Perfiles de Usuario y Roles RBAC	7
2.4	Restricciones Generales	7
2.5	Suposiciones y Dependencias	8
3.	Requerimientos Funcionales (RF)	8
3.1	Módulo de Autenticación y Cuentas	9
3.2	Módulo de Catálogo y Búsqueda	11
3.3	Módulo de Carrito de Compras	12
3.4	Módulo de Pedidos, Checkout y Pagos	13
3.5	Módulo de Tienda y Ventas (Vendedor)	15
3.6	Módulo Social, Calificaciones y Notificaciones	17
3.7	Módulo de Administración de Plataforma	18
3.8	Módulo de Mensajería y Chat	19
3.9	Módulo de Perfil y Configuración de Cuenta	20
4.	Matriz de Mapeo: Pantallas / Modales vs Endpoints API	21
5.	Requerimientos No Funcionales (RNF)	24
5.1	Seguridad	24
5.2	Rendimiento y Escalabilidad	25
5.3	Disponibilidad y Tolerancia a Fallos	25
5.4	Usabilidad y Compatibilidad	25
5.5	Integridad y Consistencia de Datos	26
5.6	Mantenibilidad y Extensibilidad	26
6.	Arquitectura Técnica y Parámetros de Infraestructura	27
6.1	Parámetros de Red y Base de Datos	27
6.2	Esquema Relacional MySQL (commercity_v2)	27
6.3	Stack Tecnológico Completo	29
6.4	Modelo de Autenticación JWT	29

6.5	Cliente HTTP Centralizado (api.js)	30
7.	Reglas de Negocio	31
8.	Diagrama de Flujo de Datos y Secuencias	32
9.	Análisis de Riesgos	33
10.	Criterios de Aceptación y Cierre de Sprint 2	34
11.	Plan de Verificación y Pruebas	35
12.	Historial de Revisiones y Control de Cambios	36
13.	Aprobaciones y Firmas	37

1. Introducción

Información del Documento	
Código	SRS-CCM-2026-V3
Proyecto	CommerCity Mobile — Marketplace Local (Android & iOS)
Organización	THE BLACK CODE JSB
Autor	Jhon Jairo Parra Obando
Versión del Sistema	v2.0 — Fase de Integración con API REST Central
Estándar	IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018
Estado	✓ Aprobado para Sprint 2
Fecha	Septiembre 2026
Revisión	3.0 — Edición Completa y Extendida

1.1 Propósito del Documento

El presente documento constituye la **Especificación de Requerimientos de Software (SRS)** oficial para el aplicativo móvil **CommerCity Mobile v2.0**, desarrollado bajo el marco de trabajo de THE BLACK CODE JSB. Su propósito es definir de manera precisa, completa, verificable y no ambigua el conjunto de requerimientos funcionales, requerimientos no funcionales, reglas de negocio, restricciones técnicas y criterios de aceptación que rigen el comportamiento del sistema.

Este documento establece el contrato técnico entre todos los actores involucrados en el desarrollo: equipo de frontend móvil, equipo de backend, diseñadores UX/UI, coordinador de casos de uso y los interesados del proyecto. Sirve como base para la elaboración de diagramas de casos de uso, diagramas de secuencia, casos de prueba y la planificación de sprints.

La versión 3.0 de este SRS incorpora la especificación completa de la transición arquitectural desde el modelo de persistencia local (prototipado) hacia el modelo real de consumo de API REST centralizada, detallando cada endpoint, contrato de datos, flujo de autenticación JWT y regla de negocio que gobierna el sistema.

 Audiencia Objetivo

Este documento está dirigido a: (1) Ingenieros de software frontend y backend, (2) Coordinadores de ingeniería y arquitectos de software, (3) Analistas y testers de calidad, (4) Directivos y stakeholders del proyecto CommerCity. La lectura técnica completa es recomendada para los roles de desarrollo; los stakeholders pueden concentrarse en las Secciones 1, 2, 7 y 10.

1.2 Alcance del Sistema

CommerCity es un marketplace móvil diseñado para dinamizar el comercio local mediante la conexión directa entre compradores y vendedores dentro de un ecosistema digital urbano. El sistema permite que comerciantes locales publiquen sus productos y gestionen sus tiendas desde un dispositivo móvil, mientras que los compradores pueden explorar, comparar y adquirir productos con soporte de múltiples métodos de pago.

El **alcance funcional** del sistema CommerCity Mobile v2.0 comprende los siguientes módulos operacionales:

- **Autenticación y Gestión de Cuentas:** Registro, inicio de sesión con JWT, recuperación de contraseña y gestión de sesiones seguras.
- **Catálogo y Exploración:** Visualización de productos por categoría, búsqueda textual, paginación y detalle de producto con stock en tiempo real.
- **Carrito de Compras Persistente:** Gestión del carrito sincronizada con el servidor, con control de stock, modificación de cantidades y expiración automática.
- **Procesamiento de Pedidos y Checkout:** Pasarela de pago con resumen fiscal calculado por el backend (IVA 19%, Comisión 10%), confirmación atómica de transacciones y comprobante de compra.
- **Panel de Tienda para Vendedores:** Gestión de inventario, publicación de productos, actualización de estados de envío y visualización de métricas financieras.
- **Mensajería y Chat:** Sistema de comunicación bidireccional entre compradores y vendedores en tiempo real.
- **Notificaciones Operacionales:** Alertas sobre estados de pedidos, mensajes nuevos y actualizaciones de plataforma.
- **Sistema de Calificaciones y Reseñas:** Valoración post-compra de vendedores con puntuación de 1 a 5 estrellas.
- **Consola Administrativa:** Panel de métricas globales, moderación de contenido y gestión de usuarios para administradores de plataforma.

Fuera del alcance de esta versión (v2.0) se encuentran: integración con pasarelas de pago externas (PSE, PayU, Stripe), logística de terceros, facturación electrónica automática DIAN, y funciones de realidad aumentada para visualización de productos.

1.3 Definiciones, Acrónimos y Abreviaturas

SRS (Software Requirements Specification)

Documento formal que describe el comportamiento esperado del software, sus restricciones y requerimientos. Estándar IEEE 830-1998.

JWT (JSON Web Token)

Estándar RFC 7519 para la transmisión segura de información entre partes como objeto JSON. Utilizado en CommerCity para autenticación sin estado (stateless).

RBAC (Role-Based Access Control)

Modelo de control de acceso basado en roles que restringe el acceso a funciones del sistema según el rol asignado al usuario autenticado.

REST (Representational State Transfer)

Estilo arquitectural para sistemas distribuidos que utiliza HTTP como protocolo de comunicación. El backend de CommerCity expone una API RESTful.

IVA (Impuesto al Valor Agregado)

Impuesto indirecto del 19% aplicado sobre el valor de los productos gravados, conforme a la normativa tributaria colombiana vigente.

Capacitor

Runtime multiplataforma de Ionic que permite empaquetar aplicaciones web (HTML/CSS/JS) como aplicaciones nativas para Android e iOS.

DBMS (Database Management System)

Sistema gestor de base de datos. En CommerCity se utiliza MySQL 8.0+ como DBMS central.

RBAC

Control de Acceso Basado en Roles. Los roles del sistema son: COMPRADOR, VENDEDOR y ADMINISTRADOR.

API (Application Programming Interface)

Interfaz de programación que expone funcionalidades del backend para ser consumidas por el cliente móvil.

CDN (Content Delivery Network)

Red de distribución de contenidos. Utilizada para almacenar y servir imágenes de productos.

HTTPS / TLS

Protocolo de transterencia segura de hipertexto con citrado Transport Layer Security. Requerido para todas las comunicaciones en producción.

Pool de Conexiones

Mecanismo de reutilización de conexiones de base de datos para mejorar el rendimiento. Implementado con `mysql2` en el backend.

Sprint

Período de trabajo de duración fija (generalmente 2 semanas) en la metodología ágil Scrum para el desarrollo iterativo del software.

1.4 Referencias y Estándares Aplicables

Referencia	Descripción	Aplicación
IEEE Std 830-1998	Práctica Recomendada para Especificaciones de Requerimientos de Software	Estructura y formato del documento SRS
ISO/IEC/IEEE 29148:2018	Sistemas e Ingeniería de Software — Procesos de Ciclo de Vida — Ingeniería de Requerimientos	Definición de calidad de requerimientos
RFC 7519	JSON Web Token (JWT) — Internet Engineering Task Force	Especificación del mecanismo de autenticación
OWASP Mobile Top 10	Guía de Seguridad en Aplicaciones Móviles	Directrices de seguridad para el cliente Capacitor
Normativa Tributaria Colombia — Ley 1819/2016	Reforma Tributaria Estructural — IVA 19%	Base legal para el cálculo del impuesto
Ionic Capacitor v5 Docs	Documentación Oficial del Runtime Multiplataforma	Empaquetado nativo Android/iOS
MySQL 8.0 Reference Manual	Documentación Oficial de MySQL	Diseño del esquema y transacciones

1.5 Visión General del Documento

Este documento está organizado en trece secciones principales siguiendo la estructura recomendada por el estándar IEEE 830-1998, adaptada a las necesidades específicas del proyecto CommerCity Mobile:

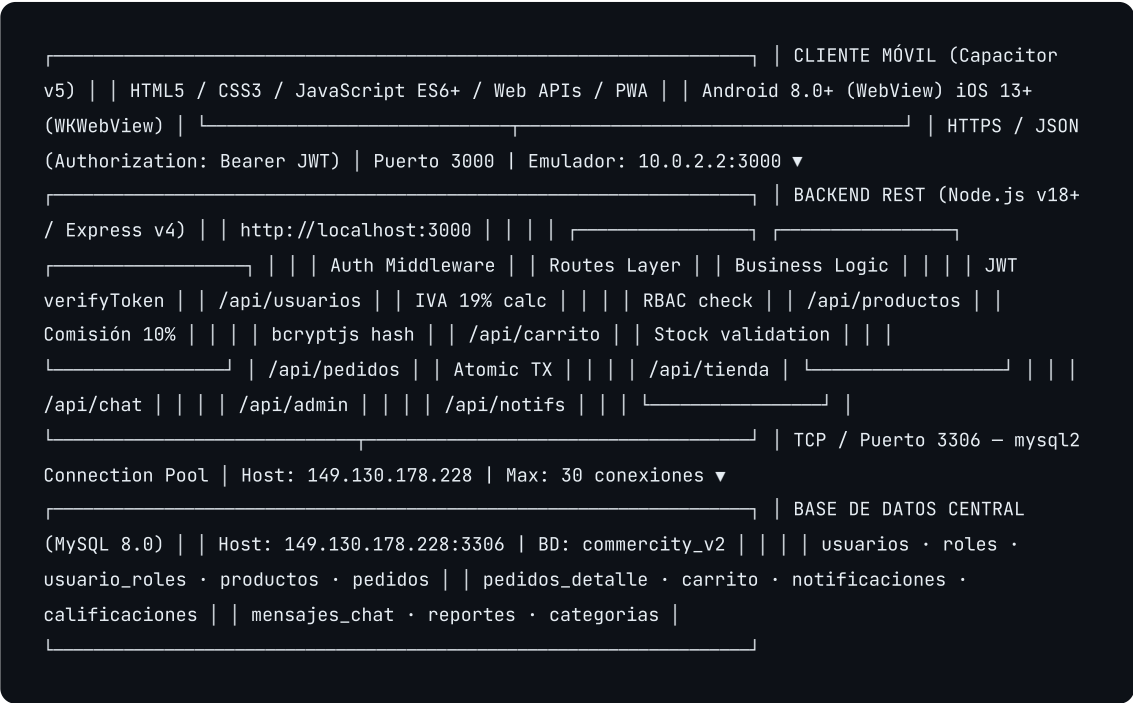
- Las **Secciones 1 y 2** proveen el contexto, alcance y descripción general del sistema.
- Las **Secciones 3 y 4** contienen la especificación detallada de todos los requerimientos funcionales y su mapeo con los endpoints de la API.
- La **Sección 5** cubre los requerimientos no funcionales de calidad del sistema.
- La **Sección 6** detalla la arquitectura técnica, esquema de base de datos y stack tecnológico.
- Las **Secciones 7 y 8** presentan las reglas de negocio y los flujos de datos.
- Las **Secciones 9 a 13** tratan el análisis de riesgos, criterios de aceptación, plan de pruebas, historial de cambios y aprobaciones.

2. Descripción General del Sistema

2.1 Perspectiva del Producto

CommerCity Mobile opera como un **cliente desacoplado de arquitectura híbrida nativa** (frontend) que se comunica con un servidor backend centralizado a través de una API REST. El sistema sigue el patrón arquitectural *Client-Server* con separación estricta de responsabilidades:

- **Cliente Móvil (Capacitor):** Responsable de la interfaz de usuario, navegación, renderizado de datos y experiencia del usuario. No realiza cálculos financieros ni validaciones de negocio críticas.
- **Backend (Node.js/Express):** Responsable de toda la lógica de negocio: autenticación, autorización por roles, cálculos fiscales, validación de inventario, gestión de transacciones y persistencia de datos.
- **Base de Datos (MySQL 8.0):** Capa de persistencia relacional que almacena todos los datos del negocio con integridad referencial garantizada.



2.2 Funciones Principales del Producto

CommerCity Mobile provee las siguientes capacidades funcionales de alto nivel:

#	Función	Descripción Resumida	Rol Beneficiado
F-01	Autenticación JWT Segura	Login real con validación en MySQL, token con expiración y control de roles	Todos
F-02	Exploración de Catálogo	Listado de productos paginado con filtros por categoría y búsqueda textual	Comprador
F-03	Carrito Persistente en Servidor	Carrito sincronizado con BD, control de stock y expiración automática a 7 días	Comprador
F-04	Checkout con Cálculo Backend	Pasarela con IVA 19% y comisión 10% calculados exclusivamente por el servidor	Comprador

#	Función	Descripción Resumida	Rol Beneficiario
F-05	Historial de Compras	Seguimiento de pedidos con estados y opción de cancelación	Comprador
F-06	Panel de Tienda	CRUD de productos, gestión de stock, actualización de envíos y métricas financieras	Vendedor
F-07	Mensajería Directa	Chat en tiempo real entre comprador y vendedor con historial persistente	Todos
F-08	Notificaciones Push	Alertas operacionales sobre pedidos, mensajes y actualizaciones del sistema	Todos
F-09	Sistema de Calificaciones	Valoración de vendedores post-compra con puntuación 1-5 estrellas y comentario	Comprador
F-10	Consola Administrativa	Dashboard global, moderación de reportes y gestión de usuarios/productos	Administrador

2.3 Perfiles de Usuario y Roles RBAC

El sistema implementa un modelo de Control de Acceso Basado en Roles (RBAC) gestionado en la tabla `usuario_rol` de la base de datos `commercity_v2`. Un usuario puede tener múltiples roles simultáneamente (por ejemplo: un usuario puede ser COMPRADOR y VENDEDOR al mismo tiempo).

Rol	ID en BD	Permisos Principales	Vistas Habilitadas
COMPRADOR	<code>rol_id = 1</code>	Navegar catálogo, gestionar carrito, realizar compras, ver historial, calificar vendedores, usar chat, reportar contenido	page-home, page-carrito, page-historial, page-perfil, page-chat, page-mensajes
VENDEDOR	<code>rol_id = 2</code>	Todo lo del comprador + publicar productos, editar stock, gestionar pedidos entrantes, ver métricas de tienda, configurar datos bancarios	page-tienda, page-pedidos, modal #add-prod-modal, modal #edit-prod-modal
ADMINISTRADOR	<code>rol_id = 3</code>	Acceso total al sistema: métricas globales, suspensión de usuarios, moderación de reportes, gestión de productos, visualización de toda la plataforma	page-admin, ajustes-admin, acceso a todos los modales de administración

2.4 Restricciones Generales

El sistema está sujeto a las siguientes restricciones de diseño e implementación:

- Sin cálculos financieros en cliente:** Ningún cálculo de IVA, comisión, subtotal o total se realizará en el código JavaScript del cliente. Todos los montos provienen exclusivamente del backend.
- Autenticación obligatoria para transacciones:** Cualquier operación que modifique datos (carrito, pedidos, productos) requiere un token JWT válido en el encabezado `Authorization: Bearer <token>`.
- Compatibilidad de plataforma:** El aplicativo debe funcionar correctamente en Android 8.0 (API 26) o superior e iOS 13 o superior.
- Puerto de Backend fijo:** El backend opera exclusivamente en el puerto 3000. En emulador Android se accede mediante la IP especial `10.0.2.2`.
- Base de datos centralizada:** No se permite persistencia local de datos críticos del negocio. Los datos maestros residen exclusivamente en MySQL `commercity_v2`.
- Lenguaje de la interfaz:** El idioma de la aplicación es español (es-CO).
- Conformidad tributaria:** El IVA se calcula conforme a la legislación tributaria colombiana vigente (19% sobre productos gravados).

2.5 Suposiciones y Dependencias

El presente SRS asume las siguientes condiciones del entorno:

- El servidor backend Node.js/Express está en ejecución y accesible en `http://localhost:3000` durante el desarrollo y pruebas.
 - La base de datos MySQL `commercity_v2` está disponible en el host `149.130.178.228:3306` con el esquema relacional completo y migrado.
 - Los dispositivos móviles objetivo tienen acceso a internet estable para comunicarse con el backend.
 - El entorno de emulador Android utiliza la convención de IP `10.0.2.2` para acceder al `localhost` del equipo anfitrión.
 - Las dependencias del backend (`bcryptjs` , `jsonwebtoken` , `mysql2` , `express` , `cors` , `dotenv`) están instaladas y actualizadas según `package.json` .
 - El módulo de carga de imágenes (multipart/form-data) dispone de almacenamiento en servidor o CDN configurado.
-

3. Requerimientos Funcionales (RF)

Los requerimientos funcionales describen *qué debe hacer el sistema* en términos de comportamiento observable, entradas aceptadas, salidas producidas y estados que debe mantener. Cada requerimiento recibe un identificador único, un nombre descriptivo, una descripción detallada, el criterio de aceptación medible, la prioridad y la trazabilidad con el endpoint de la API.

 Convención de Identificadores

Los identificadores siguen el patrón **RF-[número]** para Requerimientos Funcionales. La prioridad sigue la convención MoSCoW: M = Must Have (Debe tener), S = Should Have (Debería tener), C = Could Have (Podría tener).

3.1 Módulo de Autenticación y Cuentas

Este módulo gestiona el ciclo de vida completo de la identidad del usuario: registro, autenticación, recuperación de credenciales y cierre de sesión. Todos los flujos dependen de la API REST del backend y la base de datos `commercity_v2`.

RF-01 **Inicio de Sesión Real con JWT**

Descripción: El sistema debe autenticar al usuario enviando sus credenciales (correo electrónico y contraseña) al endpoint `POST /api/usuarios/login`. El backend verifica las credenciales contra la tabla `usuarios` de MySQL, comparando la contraseña usando `bcryptjs.compare()`. Al ser correctas, retorna un token JWT firmado con algoritmo HMAC-SHA256 con expiración de 7 días, junto con el objeto de usuario que incluye el array de roles obtenido de la tabla `usuario_rol`.

Flujo detallado:

- El usuario ingresa correo y contraseña en `page-login` y presiona "Iniciar Sesión".
- La app llama a `CommerCityAPI.auth.login(email, password)` que ejecuta `POST /api/usuarios/login` con body `{ email, password }`.
- El backend consulta: `SELECT u.id, u.nombre_completo, u.email, u.password, GROUP_CONCAT(r.nombre) as roles_str FROM usuarios u LEFT JOIN usuario_rol ur ON u.id = ur.usuario_id LEFT JOIN roles r ON ur.rol_id = r.id WHERE u.email = ?`.
- El backend retorna `{ mensaje, token, usuario: { id, nombre, email, roles: ["COMPRADOR", "VENDEDOR"] } }`.
- La app almacena el token en `localStorage` bajo la clave `commercity_auth_token` y los datos de usuario en `commercity_auth_user`.
- La función `setRole(isSeller, isAdmin)` actualiza la interfaz según los roles recibidos y navega a la pantalla correspondiente (`page-home` o `page-admin`).

Criterio de Aceptación: Un usuario con credenciales válidas obtiene token JWT, sus roles se asignan correctamente y la interfaz refleja sus permisos en menos de 2 segundos. Un usuario con credenciales inválidas recibe el mensaje "Credenciales inválidas" y permanece en la pantalla de login.

Manejo de errores: HTTP 401 → Limpiar sesión local + toast de error. HTTP 400 → Toast de campo faltante. HTTP 500 → Toast de "Error del servidor, intenta más tarde".

Prioridad: **M — Must Have**

Endpoint: `POST /api/usuarios/login`

Pantalla: `page-login`

RF-02

Registro de Nuevo Usuario

Descripción: El sistema debe permitir el registro de nuevos usuarios mediante el formulario de `page-registro`. El cliente envía los datos a `POST /api/usuarios/registro`. El backend valida que el correo no exista previamente (`HTTP 409` si ya está registrado), crea el registro en la tabla `usuarios`, asigna el rol base `COMPRADOR` en `usuario_rol` y, si el campo `solicita_vendedor` es `true`, asigna también el rol `VENDEDOR`. Finalmente, retorna un token JWT para auto-autenticar al usuario recién registrado.

Validaciones requeridas en formulario:

- Nombre completo: mínimo 3 caracteres, obligatorio.
- Correo electrónico: formato válido RFC 5322, obligatorio.
- Contraseña: mínimo 8 caracteres, obligatorio.
- Confirmación de contraseña: debe coincidir con el campo de contraseña.

Prioridad: **M — Must Have**

Endpoint: **POST /api/usuarios/registro**

Pantalla: **page-registro**

RF-03

Recuperación de Contraseña

Descripción: El sistema debe permitir al usuario solicitar el restablecimiento de su contraseña ingresando su correo electrónico registrado en `page-recuperar`. El cliente envía `POST /api/usuarios/recuperar` con `{ email }`. El backend procesa la solicitud y envía un correo con el enlace de restablecimiento al email proporcionado. La respuesta `{ exito: true, mensaje: "Instrucciones enviadas a su correo" }` se muestra mediante un toast informativo al usuario.

Restablecimiento efectivo: Al acceder al enlace del correo, `page-restablecer` permite ingresar la nueva contraseña que se envía a `POST /api/usuarios/restablecer-password` con el token de restablecimiento y la nueva clave.

Prioridad: **S — Should Have**

Endpoint: **POST /api/usuarios/recuperar**

Pantalla: **page-recuperar**

RF-04

Cierre de Sesión Seguro

Descripción: Al ejecutar el cierre de sesión, el sistema debe eliminar completamente los datos de autenticación del almacenamiento local del cliente: la clave `commercity_auth_token` y la clave `commercity_auth_user` de `localStorage`. La función `CommerCityAPI.clearSession()` gestiona esta limpieza. Posteriormente, la interfaz debe restablecer el estado de roles (`isSeller = false`, `isAdmin = false`) y navegar obligatoriamente a `page-login`.

Expiración automática de sesión: Cuando cualquier petición al backend retorna `HTTP 401` o `HTTP 403`, el interceptor de la función `request()` en `api.js` ejecuta automáticamente `clearSession()`, muestra el toast "Sesión expirada o no autorizada. Inicia sesión nuevamente." y navega a `page-login`.

Prioridad: **M — Must Have**

Acción: **localStorage.clear() + navTo('login')**

Pantalla: **Todas (header/menú)**

3.2 Módulo de Catálogo y Búsqueda

Este módulo permite a todos los usuarios explorar el catálogo de productos disponibles en la plataforma. El catálogo se alimenta en tiempo real desde la base de datos MySQL mediante el endpoint de productos.

RF-10

Consulta de Catálogo Principal

Descripción: La pantalla principal (`page-home`) debe cargar automáticamente al inicializarse el sistema el listado de productos disponibles desde `GET /api/productos` . Los productos se muestran en el grid `#home-prod-grid` y en los carruseles por categoría. El sistema debe soportar los siguientes parámetros de consulta (query params):

- `categoria` : Filtra productos por categoría (ej: "electronica", "ropa", "hogar"). Si es "todos" o está vacío, retorna todos los productos.
- `q` : Búsqueda textual por nombre o descripción del producto.
- `pagina` : Número de página para paginación (por defecto: 1).
- `limite` : Número de productos por página (por defecto: 20).

Respuesta del servidor: `{ total: 150, productos: [{ id, nombre, precio, descuento, stock, imagen_url, categoria, vendedor_nombre }] }`

Renderizado: Cada producto se muestra como una tarjeta con imagen, nombre, precio (con descuento si aplica), rating del vendedor y botón de "Agregar al carrito". Los productos con stock 0 deben mostrarse con indicador "Agotado" y el botón de carrito deshabilitado.

Prioridad: **M — Must Have**

Endpoint: `GET /api/productos`

Pantalla: `page-home`

Auth: **No requerida**

RF-11

Detalle de Producto

Descripción: Al seleccionar cualquier tarjeta de producto en el catálogo, el sistema debe abrir el modal `#pd-modal` y cargar los datos completos del producto desde `GET /api/productos/:id` . El detalle incluye: todas las imágenes del producto, descripción extendida, precio y descuento actuales, stock disponible en tiempo real, categoría, datos del vendedor (nombre, rating, cantidad de ventas), y reseñas de compradores anteriores.

Criterio de Aceptación: El modal debe abrirse y mostrar todos los datos del producto en menos de 1.5 segundos. El stock mostrado debe ser el valor actual en la base de datos al momento de la consulta.

Prioridad: **M — Must Have**

Endpoint: `GET /api/productos/:id`

Pantalla: **Modal** `#pd-modal`

RF-12

Búsqueda en Tiempo Real

Descripción: El campo de búsqueda (`#search-input`) en la pantalla principal debe permitir búsqueda textual de productos. Al escribir en el campo, el sistema debe ejecutar la consulta `GET /api/productos?q={término}` con un debounce de 400ms para evitar peticiones excesivas. Los resultados deben reemplazar dinámicamente el contenido de `#home-prod-grid` . Si la búsqueda no retorna resultados, el sistema muestra el mensaje "No encontramos productos para tu búsqueda" con una ilustración de estado vacío.

Prioridad: **S — Should Have**

Endpoint: `GET /api/productos?q=...`

Debounce: **400ms**

RF-13

Mis Productos (Vendedor)

Descripción: La vista de perfil del vendedor (`page-perfil`) debe incluir una pestaña "Mis Productos" que lista exclusivamente los productos publicados por el usuario autenticado. El sistema consulta `GET /api/productos?vendedor_id={id}` o el endpoint dedicado de la tienda, filtrando por el ID del vendedor extraído del JWT. Cada producto listado muestra stock actual, precio, estado de publicación (activo/pausado) y controles de edición.

Prioridad: **S — Should Have**

Rol requerido: **VENDEDOR**

Pantalla: `page-perfil`

3.3 Módulo de Carrito de Compras

El carrito de compras es persistente en el servidor, asociado al usuario autenticado. Toda operación sobre el carrito requiere autenticación mediante JWT y se sincroniza con la base de datos `commercity_v2` .

RF-30

Persistencia del Carrito en Servidor

Descripción: El carrito de compras del usuario autenticado debe almacenarse en la base de datos MySQL, no en `localStorage`. La pantalla `page-carrito` debe cargar los ítems actuales del carrito desde `GET /api/carrito` con el header `Authorization: Bearer <token>`. La respuesta incluye el listado de ítems: `[{ id_item, producto_id, nombre, precio_unitario, cantidad, subtotal_item, imagen_url, stock_disponible }]`.
Badges de conteo: Los contadores de ítems en el carrito (`#sidebar-cart-badge` y `#bnav-cart-dot`) deben actualizarse en tiempo real tras cada operación de agregar o eliminar producto.

Prioridad: **M — Must Have**

Endpoint: `GET /api/carrito`

Auth: **Bearer JWT requerido**

RF-31

Agregar Producto al Carrito

Descripción: Desde el modal de detalle de producto (`#pd-modal`), el usuario puede agregar el producto al carrito presionando "Agregar al Carrito". El sistema ejecuta `POST /api/carrito` con payload `{ producto_id, cantidad }`. El backend valida que haya stock disponible para la cantidad solicitada. Si el producto ya existe en el carrito del usuario, el backend puede incrementar la cantidad existente o crear un nuevo ítem según la configuración. En caso de stock insuficiente, el backend retorna HTTP 400 con el mensaje correspondiente.

Prioridad: **M — Must Have**

Endpoint: `POST /api/carrito`

Validación: **Stock en servidor**

RF-32

Modificar Cantidad de Ítems

Descripción: En `page-carrito`, los controles de cantidad (`cartQty`) permiten incrementar o decrementar la cantidad de cada ítem. Cada cambio genera una petición `PATCH /api/carrito/:id_item` con `{ cantidad: nuevaCantidad }`. El backend re-valida el stock disponible antes de aceptar el cambio. Si la cantidad solicitada supera el stock disponible, el backend retorna el error apropiado y el cliente muestra el máximo permitido. Si la cantidad llega a 0, el ítem debe eliminarse automáticamente del carrito.

Prioridad: **M — Must Have**

Endpoint: `PATCH /api/carrito/:id`

Validación: **Backend valida stock**

RF-33

Eliminar Ítem del Carrito

Descripción: El botón de eliminar (`cartRemove`) junto a cada ítem en `page-carrito` ejecuta `DELETE /api/carrito/:id_item`. El backend elimina el ítem del carrito del usuario en la base de datos. El cliente refresca el listado del carrito y actualiza los badges de conteo. Se muestra una confirmación toast "Producto eliminado del carrito".

Prioridad: **M — Must Have**

Endpoint: `DELETE /api/carrito/:id`

RF-34

Caducidad Automática del Carrito

Descripción: Los carritos sin actividad durante un período de 7 días (604.800.000 milisegundos) deben ser purgados automáticamente por el sistema. Este proceso se ejecuta en el backend mediante un job programado o trigger de base de datos que limpia los ítems de carrito cuya marca temporal de última actualización supere el umbral de inactividad. El usuario que intente acceder a un carrito purgado verá el carrito vacío con el mensaje informativo correspondiente.

Prioridad: **S — Should Have**

Período: **7 días de inactividad**

3.4 Módulo de Pedidos, Checkout y Pagos

Este módulo gestiona el proceso completo de compra, desde la validación de la dirección de entrega hasta la confirmación atómica de la transacción. Es el módulo más crítico del sistema desde la perspectiva de integridad de datos y conformidad fiscal.

⚠ Regla de Negocio Crítica — Cálculo Exclusivo en Servidor

Ningún cálculo de Subtotal, IVA (19%) o Comisión de Plataforma (10%) puede realizarse en el código JavaScript del cliente móvil. Todos los montos deben obtenerse desde el endpoint `GET`

`/api/pedidos/resumen` y renderizarse directamente en la interfaz. El código legacy de cálculo local (`const subtotal = Math.round(total / 1.19)`) está DEPRECADO y eliminado.

RF-38 Validación de Dirección de Entrega

Descripción: Antes de habilitar el acceso a la pasarela de pago (`#pasarela-modal`), el sistema debe verificar que el usuario comprador tenga al menos una dirección de entrega registrada en su perfil. Esta validación se realiza consultando los datos de perfil del usuario desde `GET /api/usuarios/perfil` . Si no existe dirección registrada, el botón "Proceder al Pago" mostrará el modal de registro de dirección o el mensaje correspondiente indicando que debe añadir una dirección antes de continuar.

Campos requeridos de la dirección: Dirección completa (calle, número), ciudad, departamento, código postal, teléfono de contacto.

Prioridad: **M — Must Have** Endpoint: `GET /api/usuarios/perfil`

RF-40 Resumen Financiero Certificado (Pasarela)

Descripción: Al abrir la pasarela de pago (`#pasarela-modal` mediante la función `openPasarela()`), el sistema debe obtener el resumen financiero oficial ejecutando `GET /api/pedidos/resumen` con el token JWT del usuario. El backend calcula y retorna el desglose completo:

- **Subtotal:** Suma de (precio_unitario × cantidad) por ítem del carrito, con descuentos aplicados.
- **IVA (19%):** Impuesto calculado sobre el subtotal gravable conforme normativa colombiana.
- **Comisión de Plataforma (10%):** Tarifa de intermediación retenida por CommerCity sobre el valor de la venta.
- **Total Definitivo:** Monto exacto a procesar por el usuario = Subtotal + IVA.

Renderizado en UI:

- `#pas-subtotal` ← `respuesta.subtotal` formateado en COP
- `#pas-iva` ← `respuesta.iva` formateado en COP
- `#pas-amount` ← `respuesta.total` formateado en COP (valor a pagar)

Nota: El campo `comision_plataforma` se muestra de manera informativa al vendedor pero no suma al total del comprador.

Prioridad: **M — Must Have** Endpoint: `GET /api/pedidos/resumen` Pantalla: **Modal #pasarela-modal**

RF-41

Confirmación Atómica de Transacción

Descripción: Al confirmar el pago en la pasarela, el sistema ejecuta `POST /api/pedidos/confirmar-pago` con el payload de la transacción. El backend procesa el pago en una transacción SQL atómica que incluye todas las siguientes operaciones como una unidad indivisible:

1. Verificar stock vigente de todos los ítems del carrito.
2. Descontar el stock de cada producto vendido en la tabla `productos`.
3. Crear el registro del pedido en la tabla `pedidos` con subtotal, IVA, comisión y total.
4. Crear los registros de detalle en `pedidos_detalle`.
5. Vaciar completamente el carrito del usuario (`DELETE FROM carrito WHERE usuario_id = ?`).
6. Registrar la notificación de "Pedido Confirmado" para el comprador y para el vendedor.

Si cualquier paso falla, toda la transacción se revierte (ROLLBACK) y el usuario recibe el mensaje de error apropiado sin pérdida de datos.

Payload de la petición:

```
{
  "metodo_pago": "tarjeta" | "nequi" | "daviplata" | "bancolombia",
  "datos_transaccion": {
    "numero_enmascarado": "4111...1111",
    "titular": "Nombre del Titular"
  },
  "direccion_entrega_id": 12
}
```

Respuesta exitosa: `{ pedido_id: 8492, comprobante: "REC-8492", estado: "APROBADO" }`

Prioridad: **M — Must Have**

Endpoint: `POST /api/pedidos/confirmar-pago`

TX: Transacción SQL Atómica

RF-42

Historial y Seguimiento de Pedidos

Descripción: La pantalla `page-historial` debe mostrar el historial completo de compras del usuario autenticado consultando `GET /api/historial/compras`. Cada pedido muestra: número de pedido, fecha, productos comprados con cantidades, total pagado y estado actual del envío. Los estados posibles son:

- **Pendiente** — Pedido confirmado, pendiente de despacho.
- **En Camino** — El vendedor actualizó el estado a "enviado".
- **Entregado** — Pedido recibido por el comprador.
- **Cancelado** — Pedido cancelado por el comprador o administrador.

La pantalla debe disponer de pestañas de filtro (`ftab`) para filtrar por estado: Todos, Pendientes, En Camino, Entregados, Cancelados.

Prioridad: **M — Must Have**

Endpoint: `GET /api/historial/compras`

Pantalla: `page-historial`

RF-43

Cancelación de Pedido

Descripción: El comprador puede solicitar la cancelación de un pedido que se encuentre en estado "Pendiente". La acción ejecuta `POST /api/historial/cancelar/:id` con el motivo de cancelación. El backend valida que el pedido pertenece al usuario autenticado y que su estado permita la cancelación. Al procesar la cancelación: (1) el estado del pedido cambia a "Cancelado", (2) el stock de los productos es restituido en la tabla `productos`, y (3) se registra la notificación correspondiente para el vendedor.

Prioridad: **S — Should Have**

Endpoint: `POST /api/historial/cancelar/:id`

Restricción: Solo pedidos en estado **Pendiente**

3.5 Módulo de Tienda y Ventas (Vendedor)

Este módulo está disponible exclusivamente para usuarios con el rol **VENDEDOR**. Provee todas las herramientas necesarias para que los comerciantes gestionen su presencia en la plataforma.

RF-50

Resumen Financiero de la Tienda

Descripción: La pantalla principal del panel de tienda (`page-tienda`) debe mostrar un dashboard con las métricas financieras del comerciante consultando `GET /api/tienda/resumen` . El backend filtra los datos exclusivamente para el usuario autenticado (vendedor). La respuesta incluye:

- **Ventas totales:** Número total de pedidos completados.
- **Ingresos brutos:** Suma total de las ventas antes de la retención de comisión.
- **Comisión retenida (10%):** Monto acumulado de comisión descontado por la plataforma.
- **Balance neto:** Ingresos brutos menos comisión retenida. Monto disponible para dispersión bancaria.
- **Pedidos pendientes:** Cantidad de pedidos en estado "Pendiente" que requieren atención.

Prioridad: **M — Must Have**

Endpoint: `GET /api/tienda/resumen`

Rol: **VENDEDOR**

RF-51

Publicación de Nuevos Productos

Descripción: El vendedor puede publicar nuevos productos mediante el modal `#add-prod-modal` . El formulario recopila: nombre del producto, descripción, precio, descuento (opcional, en %), stock inicial, categoría, e imagen del producto. La imagen se envía como `multipart/form-data` a `POST /api/productos` . El backend procesa la imagen, la almacena en el CDN/servidor y registra el producto en la tabla `productos` vinculado al `vendedor_id` del usuario autenticado.

Validaciones del formulario:

- Nombre: obligatorio, máximo 150 caracteres.
- Precio: obligatorio, valor positivo mayor a \$0.
- Stock: obligatorio, entero no negativo.
- Categoría: selección de lista predefinida.
- Imagen: obligatoria, formatos JPG/PNG/WebP, máximo 5 MB.

Prioridad: **M — Must Have**

Endpoint: `POST /api/productos (multipart)`

Pantalla: **Modal #add-prod-modal**

RF-52

Edición de Productos y Control de Stock

Descripción: El vendedor puede modificar sus productos existentes mediante el modal `#edit-prod-modal` . Los campos editables incluyen: nombre, descripción, precio, descuento, stock y estado (activo/pausado). Los cambios se envían a `PUT /api/productos/:id` con los campos modificados. El backend verifica que el producto pertenece al vendedor autenticado antes de aplicar los cambios. Las modificaciones de stock se reflejan inmediatamente en el catálogo público.

Prioridad: **M — Must Have**

Endpoint: `PUT /api/productos/:id`

Seguridad: **Backend valida propiedad del producto**

RF-53

Gestión de Pedidos Entrantes

Descripción: La pantalla `page-pedidos` muestra al vendedor todos los pedidos de sus productos que requieren gestión. Los datos se obtienen de `GET /api/tienda/pedidos?estado=pendiente` . Para cada pedido, el vendedor visualiza: nombre del comprador, dirección de entrega, productos solicitados con cantidades, total del pedido y estado actual. Desde el modal de detalle de pedido (`#od-modal`), el vendedor puede actualizar el estado a "En Camino" o "Entregado" ejecutando `PATCH /api/tienda/pedidos/:id/estado` . Este cambio de estado genera automáticamente una notificación al comprador.

Prioridad: **M — Must Have**

Endpoint: `GET /api/tienda/pedidos — PATCH /api/tienda/pedidos/:id/estado`

RF-54

Configuración de Dispersión Bancaria

Descripción: El vendedor puede registrar o actualizar sus datos bancarios para la transferencia de ganancias desde la sección de ajustes de la tienda. El sistema acepta datos de cuentas de: Bancolombia, Nequi, Daviplata. La información se almacena de forma segura en el perfil del usuario en la base de datos. Los campos requeridos incluyen: entidad bancaria, tipo de cuenta, número de cuenta/celular, nombre del titular y número de documento.

Prioridad: **S — Should Have**

Endpoint: **PUT /api/usuarios/perfil**

3.6 Módulo Social, Calificaciones y Notificaciones

RF-80

Notificaciones en Tiempo Real

Descripción: El panel de notificaciones (`#notifs-panel`) muestra al usuario autenticado sus notificaciones operacionales obtenidas de `GET /api/notificaciones` . Las notificaciones incluyen: confirmaciones de pedido, actualizaciones de envío, mensajes nuevos de chat, confirmaciones de calificación y alertas administrativas. Cada notificación muestra: ícono según tipo, título, mensaje descriptivo, fecha/hora y estado de lectura. El indicador de notificaciones sin leer aparece en el header de la aplicación.

Al abrir el panel o hacer clic en "Marcar todas como leídas", el sistema ejecuta `PATCH /api/notificaciones/marcar-leidas` con los IDs correspondientes, actualizando el estado en la base de datos y apagando el badge visual.

Prioridad: **M — Must Have**

Endpoint: **GET /api/notificaciones — PATCH /api/notificaciones/marcar-leidas**

RF-84

Calificación de Vendedores Post-Compra

Descripción: Después de recibir un pedido (estado "Entregado"), el comprador puede calificar al vendedor desde el modal `#rating-modal` . El sistema envía `POST /api/tienda/:vendedorId/calificar` con `{ puntuacion: 1..5, comentario: "texto de reseña" }` . El backend registra la calificación en la tabla `calificaciones` y recalcula el promedio de rating del vendedor. Cada usuario comprador puede calificar al mismo vendedor una sola vez por pedido.

Prioridad: **S — Should Have**

Endpoint: **POST /api/tienda/:vendedorId/calificar**

Restricción: **1 calificación por pedido**

RF-86

Reportes de Contenido Inapropiado

Descripción: Los usuarios pueden reportar productos o perfiles de usuarios que consideren inapropiados desde el modal `#report-modal` . El reporte se envía a `POST /api/reportes` con `{ entidad_tipo: "producto" | "usuario", entidad_id, motivo }` . El backend crea un ticket de moderación asignado a los administradores de la plataforma. El usuario reportante recibe confirmación de que su reporte fue enviado para revisión.

Prioridad: **S — Should Have**

Endpoint: **POST /api/reportes**

3.7 Módulo de Administración de Plataforma

Este módulo está restringido exclusivamente a usuarios con el rol **ADMINISTRADOR**. El backend verifica la presencia de este rol en el JWT antes de procesar cualquier petición al prefijo `/api/admin` .

RF-90

Dashboard de Métricas Globales

Descripción: La consola administrativa (`page-admin`) muestra un dashboard con las métricas globales de la plataforma obtenidas de `GET /api/admin/metricas` . Las métricas incluyen: total de usuarios activos registrados, total de ventas globales realizadas en la plataforma, comisión acumulada por CommerCity, número total de productos publicados, pedidos del día, lista de reportes pendientes de moderación y catálogo completo de productos para gestión. El acceso a esta pantalla está protegido en el frontend: si el usuario no tiene rol ADMINISTRADOR, la navegación a `page-admin` es bloqueada.

Prioridad: **M — Must Have**

Endpoint: `GET /api/admin/metricas`

Rol: **ADMINISTRADOR** requerido

RF-91

Moderación de Usuarios y Publicaciones

Descripción: El administrador puede gestionar el estado de cuentas de usuario y publicaciones de productos mediante `PATCH /api/admin/usuarios/:id/estado` . Las acciones disponibles son:

- **suspender:** Deshabilita el acceso del usuario a la plataforma. Las sesiones activas son invalidadas.
- **activar:** Reactiva una cuenta previamente suspendida.
- **eliminar:** Marca la cuenta como eliminada (borrado lógico). Los datos de pedidos históricos se conservan.

Estas acciones se aplican directamente en la tabla `usuarios` de MySQL y se reflejan en los intentos de autenticación subsiguientes.

Prioridad: **M — Must Have**

Endpoint: `PATCH /api/admin/usuarios/:id/estado`

3.8 Módulo de Mensajería y Chat

RF-85

Listado de Conversaciones

Descripción: La pantalla `page-mensajes` muestra al usuario autenticado todas sus conversaciones activas consultando `GET /api/chat/conversaciones` . El backend retorna las conversaciones ordenadas por la fecha del último mensaje, incluyendo para cada una: nombre e imagen del interlocutor, texto del último mensaje, fecha/hora y cantidad de mensajes no leídos (`no_leidos`). Si el usuario no tiene conversaciones previas, el sistema sugiere vendedores disponibles para iniciar un chat.

Prioridad: **S — Should Have**

Endpoint: `GET /api/chat/conversaciones`

Pantalla: `page-mensajes`

RF-85-B

Sala de Chat — Mensajes Directos

Descripción: Al acceder a una conversación específica (`page-chat`), el sistema carga el historial de mensajes mediante `GET /api/chat/mensajes/:partnerId` . El backend marca automáticamente como leídos todos los mensajes recibidos del interlocutor. Los mensajes propios se distinguen visualmente (alineación derecha, color diferente). El envío de nuevos mensajes se realiza mediante `POST /api/chat/mensajes/:partnerId` con el texto en el campo `mensaje` . Los mensajes enviados se añaden en tiempo real a la conversación.

Estructura del mensaje recibido: `{ id, emisor_id, receptor_id, mensaje, enviado_at, is_mine: boolean }`

Prioridad: **S — Should Have**

Endpoint: `GET/POST /api/chat/mensajes/:partnerId`

Pantalla: `page-chat`

3.9 Módulo de Perfil y Configuración de Cuenta

RF-95

Consulta y Edición del Perfil de Usuario

Descripción: La pantalla `page-perfil` muestra y permite editar los datos personales del usuario: nombre completo, correo electrónico, teléfono, ciudad, foto de perfil y dirección de entrega. Los datos se cargan desde `GET /api/usuarios/perfil`. Las modificaciones se envían mediante `PUT /api/usuarios/perfil` con los campos actualizados. Para vendedores, el perfil incluye adicionalmente la sección de datos bancarios para dispersión de pagos.

Prioridad: **S — Should Have**

Endpoint: `GET/PUT /api/usuarios/perfil`

4. Matriz de Mapeo: Pantallas / Modales vs Endpoints API

La siguiente matriz establece la correspondencia exacta entre cada pantalla o modal de la interfaz móvil de CommerCity y el endpoint de la API REST que consume. Esta tabla es la referencia técnica primaria para la alineación con los diagramas de casos de uso, diagramas de secuencia UML y los casos de prueba del equipo de QA.

Pantalla / Modal	Endpoint	Verbo	Auth	Payload / Params	Respuesta / Efecto
page-login	/api/usuarios/login	POST	No	{ email, password }	Retorna JWT Navega a admin
page-registro	/api/usuarios/registro	POST	No	{ nombre, email, password, solicita_vendedor }	Creación de usuario en MySQL, roles, autenticación
page-recuperar	/api/usuarios/recuperar	POST	No	{ email }	Envía correo de recuperación Toast de confirmación
page-restablecer	/api/usuarios/restablecer-password	POST	No	{ token, nuevaPassword }	Actualiza contraseña en BD. Redirige a login.
page-home	/api/productos	GET	No	?categoria=...&q=...&pagina=1	Alimenta lista de productos, carrito, categorías
Modal #pd-modal	/api/productos/:id	GET	No	Param URL: id	Carga detalles de producto: descripción, vendedor, reseñas
page-carrito	/api/carrito	GET	Bearer JWT	—	Lista items del carrito #carrito.items
Modal #pd-modal (btn agregar)	/api/carrito	POST	Bearer JWT	{ producto_id, cantidad }	Valida stock, agrega item, actualiza cantidad de carrito
cartQty en Carrito	/api/carrito/:id_item	PATCH	Bearer JWT	{ cantidad }	Actualiza cantidad de stock subtotal

Pantalla / Modal	Endpoint	Verbo	Auth	Payload / Params	Respu Efecto
<code>cartRemove</code> en Carrito	<code>/api/carrito/:id_item</code>	DELETE	Bearer JWT	—	Elimin BD. R carrito
Modal <code>#pasarela-</code> <code>modal</code>	<code>/api/pedidos/resumen</code>	GET	Bearer JWT	—	Retor <code>subtt</code> <code>comis</code> <code>}. In</code> <code>#pas</code> <code>#pas</code> <code>#pas</code>
<code>processPago()</code> en Pasarela	<code>/api/pedidos/confirmar-pago</code>	POST	Bearer JWT	<code>{ metodo_pago,</code> <code>datos_transaccion,</code> <code>direccion_entrega_id }</code>	TX at descu crea vacía Retor comp
<code>page-</code> <code>historial</code>	<code>/api/historial/compras</code>	GET	Bearer JWT	<code>?estado=todos</code>	Render histor estad por p
Cancelar en Historial	<code>/api/historial/cancelar/:id</code>	POST	Bearer JWT	<code>{ motivo }</code>	Canc restitu notifi vende
<code>page-tienda</code>	<code>/api/tienda/resumen</code>	GET	Bearer JWT (VENDEDOR)	—	Mues ingre comis balan
<code>page-pedidos</code>	<code>/api/tienda/pedidos</code>	GET	Bearer JWT (VENDEDOR)	<code>?estado=pendiente</code>	Lista client gestio despa
Modal <code>#od-</code> <code>modal</code>	<code>/api/tienda/pedidos/:id/estado</code>	PATCH	Bearer JWT (VENDEDOR)	<code>{ nuevo_estado: "En</code> <code>Camino" "Entregado" }</code>	Actua notifi comp auton
Modal <code>#add-</code> <code>prod-modal</code>	<code>/api/productos</code>	POST	Bearer JWT (VENDEDOR)	FormData: nombre, precio, stock, categoria, imagen...	Crea BD co CDN. grilla.
Modal <code>#edit-</code> <code>prod-modal</code>	<code>/api/productos/:id</code>	PUT	Bearer JWT (VENDEDOR)	FormData actualizado	Modifi produ catálo en tie
Panel <code>#notifs-</code> <code>panel</code>	<code>/api/notificaciones</code>	GET	Bearer JWT	—	Lista notifi Mues no leí
Limpiar Notificaciones	<code>/api/notificaciones/marcar-leidas</code>	PATCH	Bearer JWT	<code>{ ids: [...] }</code>	Marca BD. A visua

Pantalla / Modal	Endpoint	Verbo	Auth	Payload / Params	Respu Efecto
page-mensajes	/api/chat/conversaciones	GET	Bearer JWT	—	Lista conve activa orden último
page-chat	/api/chat/mensajes/:partnerId	GET	Bearer JWT	Param URL: partnerId	Carga mens recibi leídos
page-chat (enviar)	/api/chat/mensajes/:partnerId	POST	Bearer JWT	{ mensaje: "texto" }	Persis en BD conve tiemp
page-perfil / page-ajustes	/api/usuarios/perfil	GET / PUT	Bearer JWT	PUT: { nombre, telefono, direccion, cuenta_bancaria }	Sincro perso comm
Modal #rating- modal	/api/tienda/:vendedorId/calificar	POST	Bearer JWT	{ puntuacion: 1..5, comentario }	Alma Recal prom vende
Modal #report- modal	/api/reportes	POST	Bearer JWT	{ entidad_tipo, entidad_id, motivo }	Crea mode revisi admin
page-admin	/api/admin/metricas	GET	Bearer JWT (ADMIN)	—	Métri globa usuar comis repor
Moderación Admin	/api/admin/usuarios/:id/estado	PATCH	Bearer JWT (ADMIN)	{ accion: "suspender" "activar" "eliminar" }	Modifi del us MySQ direct

5. Requerimientos No Funcionales (RNF)

Los requerimientos no funcionales (RNF) describen *cómo debe comportarse el sistema* en términos de atributos de calidad. No describen funcionalidades específicas sino las propiedades y restricciones del sistema en su conjunto. Estos requerimientos son tan vinculantes como los funcionales y deben verificarse durante las pruebas de aceptación.

5.1 Seguridad

RNF-01 Cifrado de Comunicaciones

Todas las comunicaciones entre el cliente móvil y el servidor backend en ambiente de producción deben realizarse sobre **HTTPS con TLS 1.3**. Los certificados SSL deben ser emitidos por una Autoridad Certificadora reconocida. En desarrollo se acepta HTTP en `localhost`. Ningún dato sensible (credenciales, tokens, información financiera) puede transmitirse sin cifrado en producción.

Métrica: 100% de peticiones en producción sobre HTTPS/TLS 1.3

RNF-02 Protección de Contraseñas

Las contraseñas de usuario nunca deben almacenarse en texto plano en la base de datos. El backend utiliza `bcryptjs` con un factor de costo mínimo de 10 (2^10 iteraciones) para el hashing de contraseñas antes de almacenarlas. El cliente nunca debe almacenar contraseñas en ningún medio de persistencia local.

Librería: `bcryptjs v2.4.3+` — `bcrypt cost 10+`

RNF-03 Gestión Segura de Tokens JWT

Los tokens JWT deben firmarse utilizando el algoritmo **HMAC-SHA256 (HS256)** con una clave secreta de al menos 256 bits almacenada en variable de entorno (`.env` , nunca en código fuente). Los tokens tienen expiración de 7 días (`expiresIn: '7d'`). El cliente detecta tokens expirados (HTTP 401/403) y elimina la sesión local de inmediato. Los tokens nunca deben almacenarse en cookies sin el atributo `HttpOnly` .

Algoritmo: `HS256` Expiración: `7 días`

RNF-04 Control de Acceso Basado en Roles (RBAC)

El backend debe verificar el rol del usuario en **cada petición a endpoints protegidos**, no solo en el login. El middleware `verifyToken` debe validar la firma del JWT y extraer los roles. Los endpoints del prefijo `/api/admin` deben rechazar con HTTP 403 cualquier petición cuyo token no contenga el rol ADMINISTRADOR. Los endpoints de `/api/tienda` y `/api/productos` (POST/PUT) requieren rol VENDEDOR.

Verificación: `En cada petición (middleware)`

5.2 Rendimiento y Escalabilidad

RNF-05

Tiempo de Respuesta

Los endpoints del catálogo de productos (`GET /api/productos`) deben retornar respuesta en **menos de 350 ms** bajo carga estándar (hasta 100 usuarios concurrentes). Los endpoints de autenticación deben responder en menos de 500 ms. Las operaciones de checkout (transacciones complejas) pueden tomar hasta 2 segundos. El cliente debe mostrar indicadores de carga (spinners) para cualquier operación que supere los 300 ms.

Catálogo: **<350ms**

Auth: **<500ms**

Checkout: **<2000ms**

RNF-06

Optimización de Imágenes

Las imágenes de productos enviadas por los vendedores deben ser comprimidas en el backend antes de su almacenamiento. Las imágenes mostradas en el catálogo deben cargarse de forma optimizada (lazy loading), utilizando miniaturas para las listas y la imagen completa solo en el modal de detalle. El tamaño máximo por imagen en carga es de 5 MB.

Max tamaño: **5 MB por imagen**

Estrategia: **Lazy Loading en listas**

5.3 Disponibilidad y Tolerancia a Fallos

RNF-07

Pool de Conexiones a Base de Datos

El backend debe gestionar las conexiones a MySQL mediante un **pool de conexiones** con la librería `mysql2` . El pool debe configurarse con un máximo de **30 conexiones concurrentes**. Las conexiones inactivas deben liberarse automáticamente. En caso de pérdida de conectividad con la base de datos, el servidor debe retornar respuestas de error apropiadas (HTTP 503) sin caída del proceso Node.js.

Pool máx: **30 conexiones**

Fallback: **In-memory DB para desarrollo**

RNF-08

Manejo Graceful de Errores

El cliente móvil debe interceptar todos los errores de red y API, mostrando mensajes descriptivos al usuario sin exponer detalles técnicos internos. La función `request()` en `api.js` implementa el interceptor central. Los códigos de error deben manejarse según el siguiente esquema: 400 (solicitud incorrecta), 401/403 (sesión expirada → logout), 404 (recurso no encontrado), 500 (error del servidor). Ningún `console.error` sin manejador debe detener la aplicación.

5.4 Usabilidad y Compatibilidad

RNF-09

Compatibilidad de Plataforma

El aplicativo debe operar de manera fluida y sin errores visuales ni funcionales en:

- **Android:** Versión 8.0 (API 26) o superior. Motor de renderizado: Android WebView (Chromium).
- **iOS:** Versión 13.0 o superior. Motor de renderizado: WKWebView (WebKit).
- **Navegador de desarrollo:** Chrome 100+ / Firefox 100+ / Safari 14+.

Android min: **API 26 (v8.0)**

iOS min: **iOS 13.0**

RNF-10

Diseño Responsivo Adaptativo

El diseño de la interfaz debe adaptarse a dos breakpoints: **Mobile** (viewport ≤ 860px) para uso en dispositivos móviles en modo app, y **Desktop/Tablet** (>860px) para uso en navegador web. En modo mobile, la navegación principal es la barra inferior (`.bnav`). En modo desktop, se utiliza la barra lateral (`.sidebar`). La transición entre breakpoints debe ser fluida sin recarga de página.

5.5 Integridad y Consistencia de Datos

RNF-11 Aislamiento de Transacciones Financieras

Las transacciones de checkout deben ejecutarse con nivel de aislamiento **READ COMMITTED** en MySQL para prevenir condiciones de carrera (*race conditions*) que podrían resultar en ventas sobre inventario agotado. El backend debe implementar bloqueo optimista o pesimista según el volumen de la operación para garantizar la consistencia del stock.

Aislamiento: **READ COMMITTED**

Patrón: **Transacción atómica con ROLLBACK**

RNF-12 Precisión Fiscal

Todos los cálculos monetarios deben realizarse en el backend con precisión de centavo (2 decimales). El IVA del 19% y la comisión del 10% no deben presentar errores de redondeo que generen discrepancias entre lo mostrado al usuario y lo registrado en la base de datos. Se deben usar operaciones aritméticas de punto flotante controladas, preferiblemente con librería de decimales de precisión.

5.6 Mantenibilidad y Extensibilidad

RNF-13 Arquitectura Modular del Backend

El código del backend debe organizarse en módulos separados por dominio funcional en el directorio `/routes`: `auth.routes.js`, `products.routes.js`, `cart.routes.js`, `orders.routes.js`, `tienda.routes.js`, `chat.routes.js`, `notifs.routes.js`, `admin.routes.js`. El servidor central (`server.js`) únicamente registra los routers. El middleware de autenticación debe centralizarse en `/middleware/auth.js`.

RNF-14 Cliente HTTP Centralizado

El cliente móvil debe utilizar el módulo `CommerCityAPI` (`www/api.js`) como único punto de comunicación con el backend. Ninguna pantalla o módulo de la aplicación puede realizar peticiones directas con `fetch()` sin pasar por este wrapper centralizado. Esto garantiza la inyección automática del token JWT, el manejo uniforme de errores y la gestión centralizada de la URL base.

6. Arquitectura Técnica y Parámetros de Infraestructura

6.1 Parámetros de Red y Base de Datos

Parámetro	Valor	Observación
API Base URL (desarrollo)	http://localhost:3000	Puerto fijo, NO usar 5000 ni otros
API Base URL (emulador Android)	http://10.0.2.2:3000	IP especial para acceder al localhost del host desde AVD
DBMS	MySQL 8.0+	Servidor de producción centralizado
Host Base de Datos	149.130.178.228	Conexión exclusiva del backend, nunca expuesto al cliente
Puerto BD	3306	Puerto estándar MySQL
Nombre de Base de Datos	commercity_v2	Esquema relacional centralizado
Usuario BD	commercity_user	Credenciales gestionadas en .env
Formato de Intercambio	application/json	UTF-8 en todas las peticiones
Pool máximo de conexiones	30	Configurado en config/db.js
JWT Expiración	7 días (7d)	Configurado en auth.routes.js

6.2 Esquema Relacional MySQL (commerccity_v2)

A continuación se detalla la estructura de las tablas principales de la base de datos relacional commerccity_v2 que soporta el sistema CommerCity Mobile:

Tabla: usuarios

Campo	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador único del usuario
nombre_completo	VARCHAR(150)	NOT NULL	Nombre completo del usuario
email	VARCHAR(200)	UNIQUE, NOT NULL	Correo electrónico (identificador de login)
password	VARCHAR(255)	NOT NULL	Hash bcrypt de la contraseña
telefono	VARCHAR(20)	NULL	Número telefónico de contacto

Campo	Tipo	Restricciones	Descripción
direccion	TEXT	NULL	Dirección de entrega registrada
ciudad	VARCHAR(100)	NULL	Ciudad de residencia
foto_perfil	VARCHAR(500)	NULL	URL de la foto de perfil en CDN
cuenta_bancaria	JSON	NULL	Datos bancarios para dispersión (solo vendedores)
estado	ENUM('activo','suspendido','eliminado')	DEFAULT 'activo'	Estado de la cuenta en la plataforma
created_at	TIMESTAMP	DEFAULT NOW()	Fecha de registro

Tabla: roles

Campo	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK	ID del rol
nombre	ENUM('COMPRADOR','VENDEDOR','ADMINISTRADOR')	UNIQUE, NOT NULL	Nombre del rol del sistema

Tabla: usuario_roles

Campo	Tipo	Restricciones	Descripción
usuario_id	INT UNSIGNED	FK → usuarios.id	Referencia al usuario
rol_id	INT UNSIGNED	FK → roles.id	Referencia al rol asignado

PK compuesta: (usuario_id, rol_id) — Soporte para multirrol

Tabla: productos

Campo	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador único del producto
vendedor_id	INT UNSIGNED	FK → usuarios.id, NOT NULL	Propietario del producto
nombre	VARCHAR(150)	NOT NULL	Nombre del producto
descripcion	TEXT	NULL	Descripción extendida
precio	DECIMAL(10,2)	NOT NULL	Precio base en COP
descuento	DECIMAL(5,2)	DEFAULT 0	Porcentaje de descuento (0-100)
stock	INT UNSIGNED	NOT NULL, DEFAULT 0	Unidades disponibles en inventario
categoria	VARCHAR(80)	NOT NULL	Categoría del producto

Campo	Tipo	Restricciones	Descripción
imagen_url	VARCHAR(500)	NULL	URL de imagen en CDN
estado	ENUM('activo','pausado','eliminado')	DEFAULT 'activo'	Visibilidad en el catálogo
created_at	TIMESTAMP	DEFAULT NOW()	Fecha de publicación

Tabla: pedidos

Campo	Tipo	Descripción
id	INT UNSIGNED PK AUTO_INCREMENT	Identificador único del pedido
comprador_id	INT UNSIGNED FK → usuarios.id	Usuario que realizó la compra
subtotal	DECIMAL(12,2)	Subtotal antes de impuestos
iva	DECIMAL(12,2)	IVA calculado al 19%
comision_plataforma	DECIMAL(12,2)	Comisión CommerCity al 10%
total	DECIMAL(12,2)	Total a pagar por el comprador
estado	ENUM('Pendiente','En Camino','Entregado','Cancelado')	Estado del pedido
metodo_pago	VARCHAR(50)	Método de pago utilizado
direccion_envio	TEXT	Dirección de entrega al momento del pedido
created_at	TIMESTAMP DEFAULT NOW()	Fecha y hora de la transacción

Tabla: mensajes_chat

Campo	Tipo	Descripción
id	INT UNSIGNED PK AUTO_INCREMENT	ID del mensaje
emisor_id	INT UNSIGNED FK → usuarios.id	Usuario que envió el mensaje
receptor_id	INT UNSIGNED FK → usuarios.id	Usuario destinatario
tipo_mensaje	ENUM('texto','imagen','sistema')	Tipo de contenido del mensaje
mensaje	TEXT	Contenido del mensaje
enviado_at	TIMESTAMP DEFAULT NOW()	Fecha y hora de envío
leido	TINYINT(1) DEFAULT 0	0 = no leído, 1 = leído

6.3 Stack Tecnológico Completo

Capa	Tecnología	Versión	Rol
Frontend	HTML5 / CSS3 / JavaScript (ES6+)	—	Interfaz de usuario de la app móvil
Runtime Móvil	Ionic Capacitor	v5	Empaquetado nativo para Android e iOS
Backend Framework	Node.js + Express.js	v18+ / v4.21	API REST centralizada

Capa	Tecnología	Versión	Rol
Autenticación	jsonwebtoken (JWT)	v9.0.2	Tokens de sesión firmados HS256
Hashing	bcryptjs	v2.4.3	Protección de contraseñas
Base de Datos	MySQL	8.0+	Persistencia relacional central
Driver BD	mysql2	v3.12.0	Conexión y pool con MySQL
CORS	cors	v2.8.5	Política de origen cruzado del backend
Variables de Entorno	dotenv	v16.4.7	Configuración segura de credenciales
Build Android	Gradle	v8+	Compilación del APK nativo

6.4 Modelo de Autenticación JWT

El flujo de autenticación implementado en CommerCity sigue el patrón estándar de autenticación sin estado (stateless) con JWT:

- 1
- Solicitud de Login:** El cliente envía `POST /api/usuarios/login` con `{ email, password }` en JSON.
- 2
- Verificación en BD:** El backend consulta MySQL con JOIN en `usuario_rol`. Si el hash `bcrypt` coincide, procede a generar el token.
- 3
- Generación del Token:** `jwt.sign({ id, nombre, email, roles }, JWT_SECRET, { expiresIn: '7d' })`. El payload incluye los roles del usuario.
- 4
- Almacenamiento en Cliente:** El token se guarda en `localStorage` bajo la clave `commercity_auth_token`. El objeto de usuario en `commercity_auth_user`.
- 5
- Uso en Peticiones Protegidas:** El wrapper `CommerCityAPI.request()` en `api.js` inyecta automáticamente el header `Authorization: Bearer <token>` en cada petición.
- 6
- Verificación en Backend:** El middleware `verifyToken` en `/middleware/auth.js` valida la firma del JWT con `jwt.verify()` y extrae el payload. Adjunta el usuario a `req.user`.
- 7
- Expiración/Invalidación:** HTTP 401/403 → El interceptor de `api.js` limpia la sesión local y redirige a `page-login` con toast informativo.

6.5 Cliente HTTP Centralizado (`api.js`)

El módulo `CommerCityAPI` implementado en `www/api.js` centraliza toda la comunicación HTTP de la aplicación móvil:

```
// Detección inteligente del host según plataforma de ejecución
const BASE_URL = (window.Capacitor?.getPlatform() === 'android')
  ? 'http://10.0.2.2:3000' // Emulador Android → host machine
  : 'http://localhost:3000'; // Desarrollo web estándar

// Interceptor central: inyecta JWT y maneja errores 401/403
async function request(endpoint, options = {}) {
  const token = getToken();
  const headers = {
    'Content-Type': 'application/json',
    ...(token ? { 'Authorization': `Bearer ${token}` } : {}),
    ...options.headers
  };

  const res = await fetch(`${BASE_URL}${endpoint}`, { ...options, headers });

  if (res.status === 401 || res.status === 403) {
    clearSession(); // Elimina token y datos locales
  }
}
```

```
showToast('Sesión expirada. Inicia sesión nuevamente.', 'error');
navTo('login');          // Redirige a pantalla de login
return null;
}
// ...
}
```

La API expone métodos de conveniencia agrupados por dominio: `CommerCityAPI.auth`, `CommerCityAPI.productos`, `CommerCityAPI.carrito`, `CommerCityAPI.pedidos`, `CommerCityAPI.tienda`, `CommerCityAPI.chat`, `CommerCityAPI.notificaciones`, `CommerCityAPI.admin`.

7. Reglas de Negocio

Las reglas de negocio definen las políticas y restricciones que rigen el comportamiento del sistema en términos comerciales y operativos. Son invariantes que deben mantenerse independientemente de la capa técnica que las implemente.

ID	Regla de Negocio	Descripción	Implementación
RN-01	Cálculo Exclusivo en Backend	Ningún cálculo de IVA (19%), comisión (10%) o total de orden puede realizarse en el cliente móvil. Todos los montos financieros provienen del backend.	Endpoint <code>GET /api/pedidos/resumen</code>
RN-02	IVA del 19%	El Impuesto al Valor Agregado se aplica a todos los productos gravados según la normativa tributaria colombiana (Ley 1819/2016). El IVA es responsabilidad legal del vendedor, retenido y reportado por la plataforma.	Cálculo en backend: <code>iva = subtotal * 0.19</code>
RN-03	Comisión de Plataforma 10%	CommerCity retiene el 10% del valor bruto de cada transacción completada como comisión de intermediación. Este monto se descuenta del balance neto del vendedor en la liquidación.	Cálculo en backend: <code>comision = subtotal * 0.10</code>
RN-04	Stock No Negativo	El stock de un producto nunca puede ser negativo. Si una transacción intenta descontar más unidades de las disponibles, la transacción es rechazada con HTTP 400 y se hace ROLLBACK de la operación completa.	Validación en backend antes de SQL UPDATE
RN-05	Carrito Expirado a 7 Días	Los ítems de carrito sin actividad por más de 7 días consecutivos son eliminados automáticamente del sistema. El usuario es notificado si intenta acceder a un carrito expirado.	Job programado en backend
RN-06	Asignación Automática de Rol COMPRADOR	Todo usuario nuevo registrado en la plataforma recibe automáticamente el rol COMPRADOR como rol base, independientemente de si también solicita el rol VENDEDOR.	Lógica en <code>POST /api/usuarios/registro</code>
RN-07	Cancelación Solo en Estado Pendiente	Un pedido solo puede ser cancelado por el comprador cuando su estado es "Pendiente". Los pedidos "En Camino" o "Entregados" no admiten cancelación directa por el cliente.	Validación en <code>POST /api/historial/cancelar/:id</code>
RN-08	Restitución de Stock en Cancelación	Al cancelar un pedido, el stock de todos los productos incluidos debe restituirse automáticamente en la tabla <code>productos</code> en la misma transacción SQL de cancelación.	Transacción SQL en backend
RN-09	Una Calificación por Pedido	Un comprador solo puede registrar una calificación al vendedor por cada pedido completado ("Entregado"). No se permiten calificaciones múltiples para el mismo pedido.	Restricción UNIQUE en BD
RN-10	Propiedad de Productos	Un vendedor solo puede editar o eliminar productos que él mismo publicó. El backend	Validación en backend: <code>WHERE id = ? AND vendedor_id = ?</code>

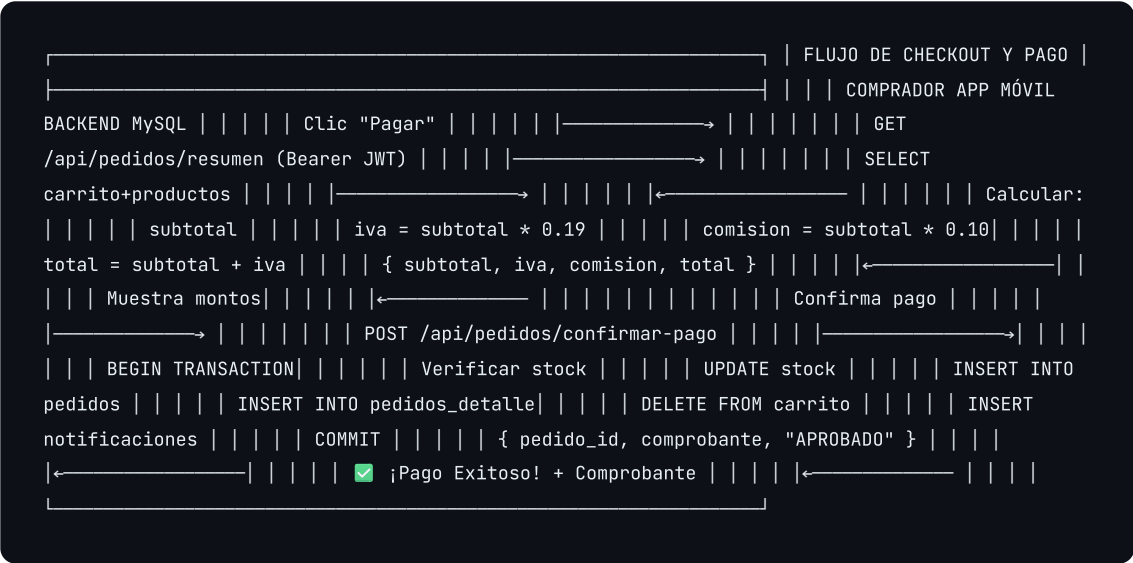
ID	Regla de Negocio	Descripción	Implementación
		verifica que el <code>vendedor_id</code> del producto coincida con el <code>id</code> del usuario autenticado en el JWT.	
RN-11	Correo Electrónico Único	El correo electrónico de cada usuario debe ser único en el sistema. No se permiten registros duplicados de email. El intento de registro con un email existente retorna HTTP 409 (Conflict).	Restricción UNIQUE en columna <code>email</code>
RN-12	Dirección Requerida para Checkout	El proceso de checkout no puede completarse si el comprador no tiene una dirección de entrega registrada en su perfil. El sistema bloquea la pasarela de pago y solicita el registro de la dirección.	Validación en frontend + backend

8. Diagrama de Flujo de Datos y Secuencias

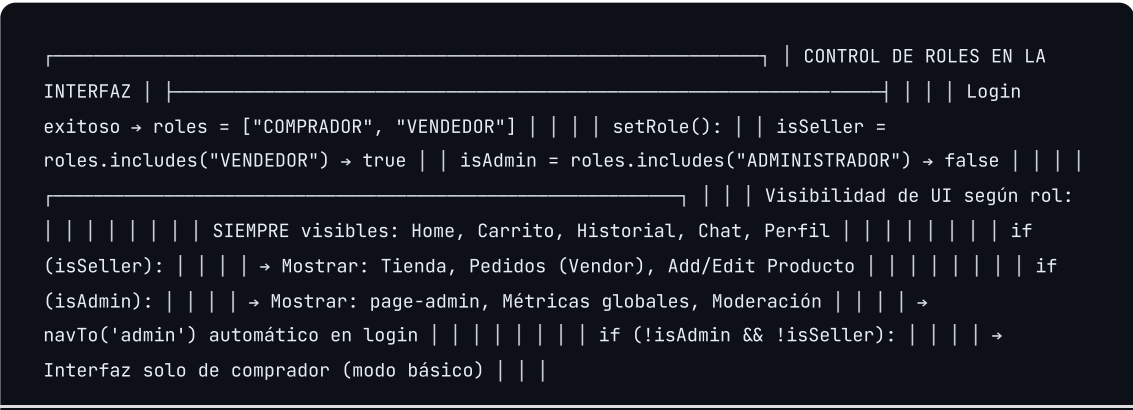
8.1 Flujo de Autenticación



8.2 Flujo de Checkout y Confirmación de Pago



8.3 Flujo de Rol RBAC en el Frontend





9. Análisis de Riesgos

El siguiente análisis identifica los principales riesgos técnicos y de negocio asociados al desarrollo y operación de CommerCity Mobile v2.0, junto con las estrategias de mitigación propuestas.

ID	Riesgo	Impacto	Probabilidad	Nivel	Mitigación
R-01	Race Condition en Stock	Alto	Media	Alto	Transacciones SQL con nivel READ COMMITTED y bloqueo de filas durante el checkout.
R-02	Pérdida de Conectividad BD	Alto	Baja	Alto	Pool de conexiones con reconexión automática. Fallback a in-memory DB en desarrollo. Respuestas HTTP 503 controladas.
R-03	Token JWT Comprometido	Alto	Baja	Medio	Expiración de 7 días. HTTPS en producción. Clave secreta JWT en variables de entorno (.env), nunca en código fuente.
R-04	Elevación de Privilegios	Alto	Baja	Medio	Verificación de roles en cada petición del backend (middleware). El cliente solo oculta UI pero el backend rechaza operaciones no autorizadas.
R-05	Inyección SQL	Alto	Baja	Medio	Uso de queries parametrizadas con <code>mysql2</code> . Nunca concatenación de strings en queries SQL.
R-06	Discrepancia de Cálculos Fiscales	Medio	Media	Medio	Centralización de todos los cálculos en el backend. Eliminación del código legacy de cálculo en cliente.
R-07	Carga de Imágenes Pesadas	Bajo	Alta	Bajo	Validación de tamaño máximo (5 MB) en cliente y backend. Compresión automática en el servidor. Lazy loading en listas.
R-08	Compatibilidad de Versiones Android/iOS	Medio	Baja	Bajo	Pruebas en emuladores de versiones mínimas soportadas (Android 8.0 / iOS 13). Testing en Capacitor.

10. Criterios de Aceptación y Cierre de Sprint 2

Los siguientes criterios deben ser verificados y aprobados para considerar el Sprint 2 como completado. Cada criterio es medible y verificable por el equipo de desarrollo y el coordinador del proyecto.

#	Criterio de Aceptación	Método de Verificación	Estado
CA-01	Autenticación Real JWT: El login consume exclusivamente <code>POST /api/usuarios/login</code> . No existe ningún código de autenticación mock (credenciales hardcodedas, roles por email) en producción.	Revisión de código + test de login con usuario real en MySQL	Pendiente
CA-02	Cero Cálculos Financieros en Cliente: La pasarela de pago obtiene subtotal, IVA y total exclusivamente del endpoint <code>GET /api/pedidos/resumen</code> . No existe código de cálculo local de IVA o comisión en <code>app.js</code> activo.	Revisión de código + interceptar petición de red en DevTools para verificar valores	Pendiente
CA-03	Control RBAC Funcional: La interfaz refleja correctamente los permisos según los roles retornados por la BD. Un usuario solo COMPRADOR no puede acceder a <code>page-tienda</code> ni <code>page-admin</code> .	Prueba con usuarios de diferentes roles. Verificar que el backend rechaza peticiones con rol inadecuado (HTTP 403).	Pendiente
CA-04	Conectividad BD Verificada: El backend conecta exitosamente a <code>commercity_v2</code> en <code>149.130.178.228:3306</code> . La función <code>isLiveDb()</code> retorna <code>true</code> en el entorno de integración.	Ejecutar <code>node test-api.js</code> y verificar respuestas reales de MySQL	Pendiente
CA-05	Checkout Atómico: La confirmación de pago ejecuta correctamente la transacción SQL atómica: descuento de stock + creación de pedido + vaciado de carrito en una sola operación ACID.	Prueba de compra completa. Verificar en MySQL que stock fue descontado y pedido creado. Probar caso de fallo (stock insuficiente) para verificar ROLLBACK.	Pendiente
CA-06	Cobertura de Endpoints: El 100% de los endpoints listados en la Sección 4 (Matriz de Mapeo) tienen al menos una pantalla o modal que los consume y una respuesta verificable.	Revisión de la matriz de mapeo contra el código de rutas del backend	Pendiente
CA-07	Alineación con Casos de Uso: Los identificadores de pantallas y modales del SRS coinciden 1:1 con los diagramas de secuencia y casos de uso elaborados por el coordinador (Yepes), permitiendo la trazabilidad completa de requerimientos.	Revisión cruzada entre este SRS y los diagramas UML del equipo	Pendiente

11. Plan de Verificación y Pruebas

11.1 Tipos de Pruebas

Tipo	Descripción	Herramientas	Cobertura Objetivo
Pruebas Unitarias de Backend	Verificación de cada route handler de forma aislada con mocks de BD	Jest / Mocha	Módulos auth, productos, carrito
Pruebas de Integración API	Pruebas end-to-end de todos los endpoints con BD real de staging	Postman / Insomnia / curl	100% de endpoints del SRS
Pruebas de Seguridad	Verificación de RBAC: intentar acceder a endpoints admin/vendedor sin rol correspondiente	Manual + Postman	Todos los endpoints protegidos
Pruebas de Interfaz (UI)	Verificar renderizado correcto de datos en cada pantalla y modal	Chrome DevTools / Emulador Android	Todas las pantallas del SRS
Pruebas de Carga	Simular 100 usuarios concurrentes accediendo al catálogo	Artillery / k6	Endpoints de alta demanda
Pruebas de Regresión	Verificar que cambios en una funcionalidad no rompen otras	Suite automatizada	Módulos críticos: auth, checkout

11.2 Casos de Prueba Prioritarios

ID Caso	Módulo	Escenario	Resultado Esperado
CP-001	Auth	Login con credenciales válidas de usuario MySQL	HTTP 200, token JWT en respuesta, navegación a home
CP-002	Auth	Login con contraseña incorrecta	HTTP 401, toast de error, permanece en login
CP-003	Auth	Petición a endpoint protegido sin token	HTTP 401, clearSession(), navTo('login')
CP-004	RBAC	Usuario COMPRADOR accede a <code>/api/tienda/resumen</code>	HTTP 403 Forbidden del backend
CP-005	Carrito	Agregar producto con stock disponible	HTTP 201, ítem en carrito, badge actualizado
CP-006	Carrito	Agregar producto con stock = 0	HTTP 400, toast "Sin stock disponible"
CP-007	Checkout	Confirmar pago con datos válidos	HTTP 200, pedido creado, carrito vaciado, stock descontado
CP-008	Checkout	Verificar que <code>#pas-iva</code> muestra valor del backend	El valor mostrado = <code>subtotal * 0.19</code> del servidor
CP-009	Checkout	Checkout con stock agotado en concurrencia	HTTP 400 + ROLLBACK, stock no modificado

ID Caso	Módulo	Escenario	Resultado Esperado
CP-010	Chat	Enviar mensaje entre comprador y vendedor	HTTP 201, mensaje persiste en MySQL, aparece en sala de chat
CP-011	Admin	Suspender usuario con rol ADMINISTRADOR	HTTP 200, estado en BD cambia a 'suspendido'
CP-012	Vendedor	Publicar producto con imagen (multipart)	HTTP 201, producto en BD con URL de imagen en CDN

12. Historial de Revisiones y Control de Cambios

Versión	Fecha	Autor	Descripción de Cambios
1.0	Mayo 2026	Equipo CommerCity	Versión inicial del SRS. Documentación de la arquitectura de prototipo con persistencia local (localStorage, datos mock).
2.0	Agosto 2026	Jhon Jairo Parra Obando	Actualización completa para Sprint 2. Incorporación de especificación de integración con API REST. Definición de endpoints, contratos de datos y eliminación de lógica mock. Adición de matriz de mapeo pantallas-endpoints.
3.0	Septiembre 2026	Jhon Jairo Parra Obando	Edición completa y extendida. Adición de secciones: reglas de negocio, análisis de riesgos, plan de pruebas, criterios de aceptación, esquema detallado de base de datos, flujos de autenticación, diagramas de secuencia ASCII, stack tecnológico completo y página de aprobaciones. Incorporación del estándar ISO/IEC/IEEE 29148:2018.

13. Aprobaciones y Firmas

El presente documento de Especificación de Requerimientos de Software (SRS-CCM-2026-V3) requiere la aprobación formal de los siguientes roles antes de proceder con la implementación de los módulos especificados:

Elaborado por

Responsable de la documentación de requerimientos, arquitectura técnica y redacción del SRS.

Jhon Jairo Parra Obando

Autor del SRS / Desarrollador Principal
THE BLACK CODE JSB

Fecha: Septiembre 2026

Revisado por

Coordinador de Casos de Uso y diagramas de secuencia.
Responsable de la trazabilidad con UML.

Coordinación de Arquitectura

Coordinador de Casos de Uso / UML
THE BLACK CODE JSB

Fecha: _____

Aprobado por

Director técnico del proyecto. Aprueba la especificación para su implementación en Sprint 2.

Dirección Técnica

Director Técnico del Proyecto
THE BLACK CODE JSB

Fecha: _____

Verificado por QA

Equipo de calidad. Confirma que los criterios de aceptación son verificables y medibles.

Equipo de Calidad

QA / Testing Lead
THE BLACK CODE JSB

Fecha: _____

 **Nota Legal**

Este documento es propiedad de THE BLACK CODE JSB y contiene información confidencial del proyecto CommerCity. Queda prohibida su reproducción, distribución o divulgación a terceros sin autorización expresa por escrito del autor responsable (Jhon Jairo Parra Obando). Uso estrictamente interno del equipo de desarrollo.